

COMPREHENSIVE TECHNICAL REPORT ON THE DUNGEON MASTER BOT SYSTEM: ARCHITECTURE, MEMORY STRATEGY, AND RETRIEVAL-AUGMENTED GENERATION (RAG) IMPLEMENTATION

Authored By:

Shreyansh Kumar
Shourya Agarwal

October 2025

EXECUTIVE SUMMARY OF KEY SYSTEM SPECIFICATIONS

System Architecture	Three-Tier (Presentation, Business Logic, Data)
Core Framework	LangChain Expression Language (LCEL)
Language Model (LLM)	ChatGroq API (llama-3.1-8b-instant, temp 0.8)
RAG Methodology	Multi-Source RAG (STM, LTM, Character Memory)
Embedding Model	HuggingFace all-MiniLM-L6-v2 (384 dimensions)
Data Persistence	ChromaDB Vector Stores (3 separate instances)
User Interface	Streamlit Orchestrator (app.py)

Dungeon Master Bot Overview

Purpose and Scope

This document provides a high-level introduction to the Dungeon Master Bot, an AI-powered interactive storytelling application that simulates a tabletop role-playing game experience. This overview covers the system's capabilities, architecture, and core components.

Table of Contents:

- System Capabilities/Features
- System Architecture Overview
- Core Components
- AI Processing Pipeline
- Memory Architecture Summary
- RAG Pattern Overview
- Technology Stack
- Performance Profile

System Capabilities

The Dungeon Master Bot provides an interactive narrative experience with the following capabilities:

Capability	Description	Implementation
Dynamic Storytelling	Generates contextual narrative responses to player actions	LLM-powered generation with retrieval-augmented context
Persistent Memory	Maintains long-term narrative consistency across sessions	Multi-tiered memory with vector search
Character Tracking	Remembers NPC personalities and relationship history	Isolated character-specific vector database

Capability	Description	Implementation
Quest Management	Detects and tracks achievements and objectives	LLM-based quest extraction with dedicated storage
Contextual Awareness	Recalls relevant past events for narrative continuity	Semantic search across historical summaries

The system operates as a conversational agent where:

1. Players input text actions describing what they want to do
2. The AI generates 1-2 sentence story continuations with action suggestions
3. All interactions are compressed and stored for future retrieval
4. Character relationships and quests are tracked automatically

System Architecture Overview

Component Hierarchy

The application consists of four Python modules organized in a three-tier architecture:



Core Components

1. Streamlit UI (app.py)

The application entry point that:

- Initializes all memory systems via `init_memory()`
- Renders the chat interface with message history
- Handles special 'q' command for quest display
- Orchestrates the turn lifecycle

Key Session State Variables:

- `messages` : Full chat history for display (append-only list)
- `short_term_memory` : Sliding window of last 8 messages (4 turns)
- `retriever` : Long-term memory vector search interface
- `char_retriever` : Character-specific memory search interface
- `quest_retriever` : Quest storage search interface

2. Backend Module (backend. py)

Contains five core functions that power the AI logic:

Function	Purpose	LLM Chain
<code>init_memory(sess)</code>	Initializes session state and ChromaDB instances	N/A
<code>gen_story(text, hist, ret, char_ret)</code>	Main narrative generation with RAG	y
<code>find_npc(text)</code>	Extracts character names from text	y
<code>sum_turn(text, resp)</code>	One-sentence turn summarization	y
<code>sum_npc_turn(name, text, resp)</code>	Character-specific relationship summary	y

Initialization Process:

1. Creates opening narrative message
 2. Initializes empty short_term_memory list
 3. Creates ChromaDB instance for long-term memory with k=3 retrieval
 4. Initializes character and quest retrievers via module imports
-

3. Character Memory Module (character_memory.py)

Manages NPC-specific memories in an isolated vector database:

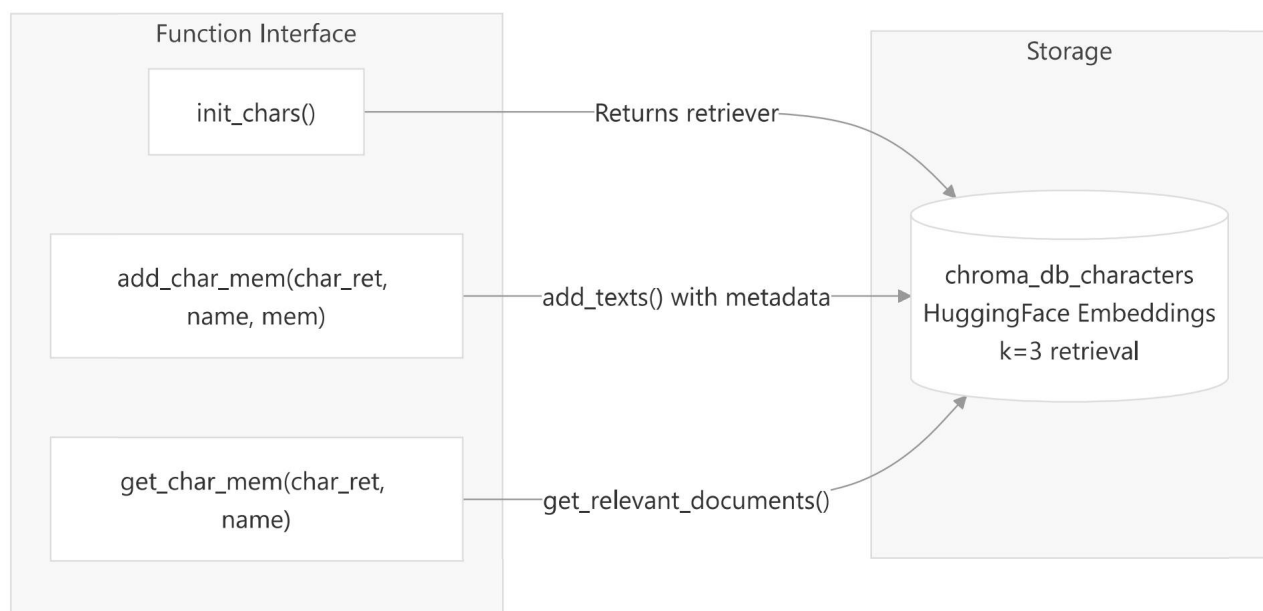


Diagram: Character Memory Operations Flow

Metadata Structure: Each character memory is stored with {"character": name} metadata
character_memory.py is enabling character-filtered retrieval.

4. Quest System (quest.py)

Detects and tracks achievements using LLM-based extraction:

Detection Logic:

1. LLM prompt requests 5-word-or-less quest summary
2. Response is validated for "none" content
3. Valid quests are stored in chroma_db_quests

Retrieval Method: get_quests() bypasses semantic search and returns all stored quests via direct Collection access quest.py 38

AI Processing Pipeline

LLM Chain Architecture

All AI operations use the LangChain Expression Language (LCEL) pattern:



Diagram: Standard LLM Chain Pattern

Shared AI Components:

- LLM: ChatGroq with llama-3.1-8b-instant model at temperature 0.8
- **Embeddings:** HuggingFace ai-MiniLM-L6-v2 generating 384-dimensional vectors

Memory Architecture Summary

Multi-Tiered Storage Strategy

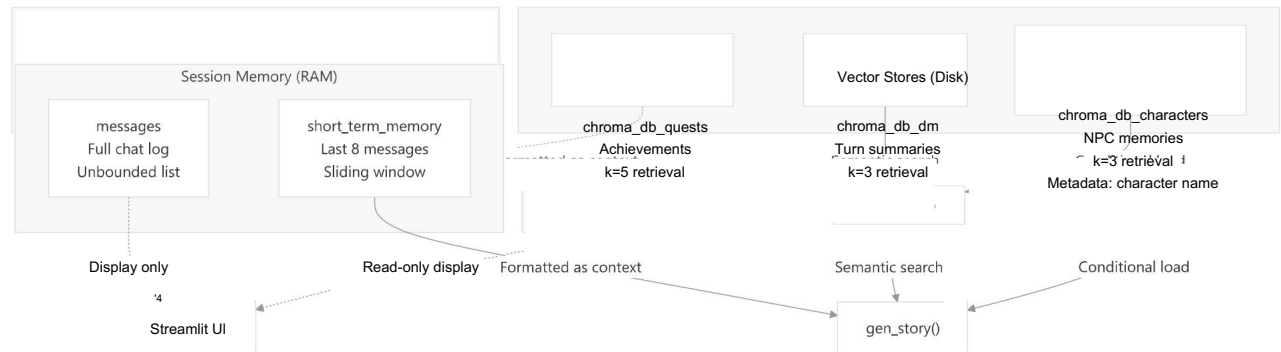


Diagram: Memory Tier Relationships and Data Flow

Memory Update Sequence

Step	Memory Tier	Operation	Code Location
1	messages	Append user + AI messages	app.py 28
2	short_term_memory	Extend with turn pair	app.py 51-54
3	short_term_memory	Trim to Last 8 messages	app.py 57-58
4	chroma_db_dm	Add turn summary	app.py 63
5	chroma_db_characters	Add NPC summary (conditional)	app.py 71
6	chroma_db_questions	Add quest (conditional)	app.py 74

RAG Pattern Overview

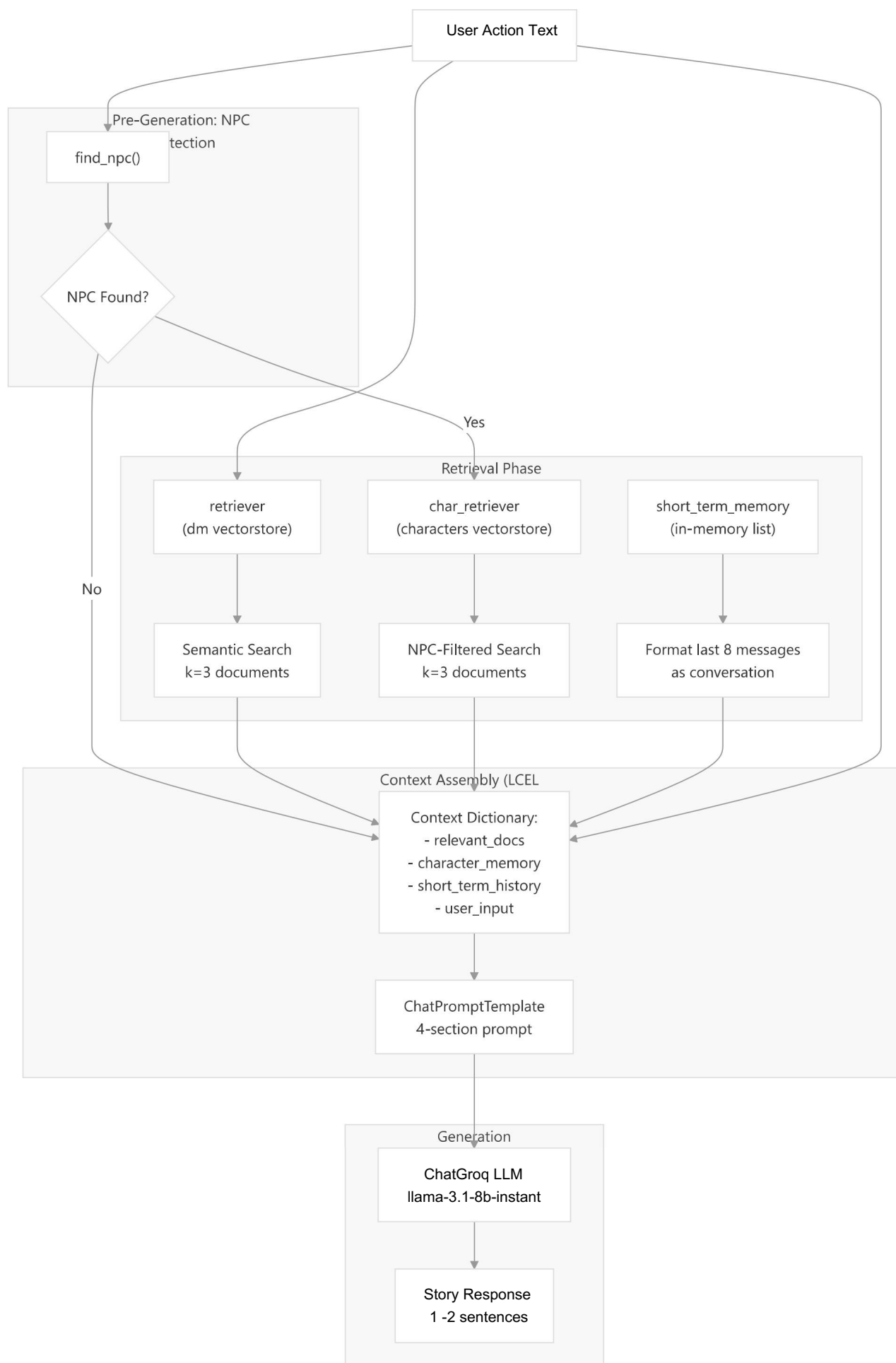
The system implements a **multi-source RAG architecture** where story generation queries three distinct context sources simultaneously:

Context Source	Type	Retrieval Method	Purpose
Long-Term Memory	Vector Store	Semantic similarity (k=3)	Historical narrative consistency
Character Memory	Vector Store	NPC-filtered similarity (k=3)	Character personality and relationships
Short-Term Memory	In-Memory List	Direct array access (last 8 messages)	Immediate conversational context

Unlike traditional single-source RAG implementations, this architecture enables **domain-specific retrieval** where character-specific context is isolated from general world knowledge, preventing NPC personality bleed across different characters.

The RAG pattern executes in three distinct phases:

1. **Pre-Generation NPC Detection:** The `find_npc()` function analyzes user input to determine if character-specific retrieval is needed
2. **Parallel Retrieval:** Multiple retrievers execute semantic searches concurrently while short-term memory is formatted
3. **Context Assembly:** Retrieved documents are merged into a structured prompt template with explicit section markers



Technology Stack

Component	Technology	Purpose	Configuration
UI Framework	Streamlit	Chat interface and state management	Session state for persistence
LLM Provider	ChatGroq API	Text generation and extraction	llama-3.1-8b-instant , temp=0.8
Embeddings	HuggingFace Transformers	Text vectorization	all-MiniLM-L6-v2,384 dims
Vector Store	ChromaDB	Semantic search and persistence	Three separate databases
Orchestration	LangChain	LLM chain composition	LCEL pattern throughout

System Characteristics

Performance Profile

- **Lines of Code:** ~250 total across 4 modules
- **LLM Calls per Turn:** 1-4 depending on NPC presence and quest detection
- **Memory Footprint:** Session state + 3 ChromaDB instances
- **Retrieval Configuration:** k=3 for DM and character memory, k=5 for quests